Implementation plan

1 Project description

1.1 Name of the project

Smart Desk Guardian (SDG): an IoT device that monitors workspace ergonomics and wellness.

1.2 Goal of the project

SDG helps reduce digital eye strain, sitting too close to the screen and long sitting periods for computer users. An Arduino Uno WiFi Rev2 measures screen distance, ambient light and heart rate, and the user presses a button when they sit down to start the sitting timer. The Arduino warns the user on the spot with a Grove-LCD RGB Backlight message and a buzzer when a threshold is crossed: closer than 50 cm to the screen, seated for more than 45 minutes, a room too dark for screen work, or a heart rate above the user's resting baseline during an ECG check.

The Arduino publishes all readings over WiFi using MQTT. A Raspberry Pi acts as the project's local server: it runs the Mosquitto MQTT broker, stores readings and session history in a database, and keeps the user accounts (identified by username/password or RFID) so a returning user is recognised. The custom app connects to the broker and shows live values, alerts and session history. Local warnings run on the Arduino, so they work even when the app, the Pi or the network is offline.

Data flow: sensors and button → Arduino Uno WiFi Rev2 (reads and filters, alert logic, Grove-LCD RGB Backlight, buzzer) → WiFi, MQTT, JSON → Raspberry Pi (Mosquitto MQTT broker, database, user accounts) → MQTT over WebSockets → custom app.

1.3 Hardware used in the project

Arduino Uno WiFi Rev2 + Grove Base Shield, Raspberry Pi with SD card, Grove Ultrasonic Ranger, Grove button (replaces the pressure sensor to start the sitting timer), DFRobot SEN0213 ECG heart rate sensor (connected with a Grove-to-jumper cable, since its 3-pin Gravity connector does not fit Grove cables), Grove light sensor, Grove-LCD RGB Backlight, Grove buzzer and cables from the Grove Starter Kit.

2 Schedule

2.1 How the project is progressing

The project runs 61 days (about 9 weeks), with a teacher review meeting every week. In Week 1 the team agrees on the MQTT topics and JSON format, so the Arduino, Pi and app can be built in parallel. Each part is tested on its own first, then integrated end to end in Week 7. Feature freeze is at the end of Week 8.

2.2 Milestones

Week	Milestone	Teacher review
1	Plan approved; lab inventory checked (ECG, light sensor, button); Git repo; MQTT topics and JSON format agreed	Plan approval
2	Ultrasonic sensor and button wired and tested on the Arduino; app wireframes	Serial monitor demo
3	ECG and light sensors added; Arduino sends all readings as JSON; app skeleton	Complete sensor node
4	Arduino connects over WiFi and publishes to the Mosquitto broker on the Pi; Pi stores readings in the database	Messages visible in an MQTT client
5	Grove-LCD RGB Backlight and buzzer alert logic on the Arduino; app shows live values	Local alerts demo
6	App alerts screen, session history and user login	App demo with live data
7	End-to-end integration; false-alarm filtering	Full system demo
8	Enclosure, stability test, bug fixes; feature freeze	Prototype inspection
9	Slides, backup demo video, final demo	Final grading and seminar

3 Resources

3.1 Who does what?

The project is budgeted at 150 hours, 50 per person (about 5.5 hours a week each). At a student engineering rate of €20/hour, that is €3,000 of labor.

Task	Lead	Hours
Project management, weekly reports, documentation	Mehdi Tahari	15
Arduino firmware, sensor and button wiring, calibration, WiFi MQTT publishing	Mehdi Tahari	23
Raspberry Pi setup and Mosquitto broker (MQTT and WebSockets)	Angel Velazquez	15
MQTT topics, credentials, connection reliability and database (readings, session history, user accounts)	Angel Velazquez	15
Grove-LCD RGB Backlight and buzzer alert logic (Arduino)	Angel Velazquez	8
App UI design (screens, alert flow)	Adam Azizi	13

App development: live dashboard, alerts, history	Adam Azizi	25
Integration, testing, enclosure and seminar	All three	36
Total		150

3.2 Who is the project manager?

Mehdi Tahari

3.3 What other roles are in the project?

• Angel Velazquez, IoT gateway engineer: Raspberry Pi, MQTT broker, database and user accounts, Grove-LCD RGB Backlight and buzzer alert logic.

• Adam Azizi, app developer: design and build of the custom app that shows live data, alerts and history.

4 Risks

4.1 If there are uncertainties at the moment?

The ECG sensor is available, but its signal is noisy when the user moves. Heart rate is measured during a short ECG check (the user sits still for about 15 to 30 seconds) instead of continuously.
The user may forget to press the button when sitting down or leaving. The ultrasonic sensor is used as a cross-check: if nobody is detected at the desk for a few minutes, the sitting timer stops.
The school WiFi may block MQTT traffic. Local alerts still work on the Arduino, and the Raspberry Pi can run its own WiFi hotspot as a fallback.